

```
1-p probability that t does not index a relevant document
1-q probability that t does not index a non-relevant
document
```

The term weight would then be given by the following equation:

$$w(t) = \log(p(1-q)/(q(1-p)))$$

To give real-world values to the term weight, it is expressed in terms of  $N$ —the document collection size,  $R$ —the subset of relevant documents that are relevant for a search term  $t$ ,  $n$ —the number of documents indexed by term  $t$ ,  $r$ —the number of indexed documents that are relevant, and some constants that are chosen for optimal results. That is as far as we go here.

Like the vector space model, this model too calculates weights for documents that are indexed by the query terms. In the vector space model, the cosine similarity acts as a surrogate for this weight. But unlike for the vector space model, the document weights can keep changing based on user interaction. This model starts with a basic set of ‘relevant’ documents in the result set, using in-document term counts or inverse document frequency (see *term weighting* in the Jargon Buster) to calculate document weights. But the model expects the user to select the more relevant documents in this set, and resubmit the query, thereby allowing the model to reformulate the query, recalculate document weights, and revise the result set. In other words, the search process is iterative if it has to give increasingly meaningful results. For first-hand experience of this model at work, try [4] with search-related queries.

To get a closer look at where the rubber meets the road, you can read how Lucene—a search engine from the Apache stables—ranks results [5]. If you try out Lucene (or a similar search engine), you might get a glimpse into how the best of algorithms cannot avoid giving you counter-intuitive result rankings, or how sometimes, unlikely key-words can give you relevant results.

Is the quest for the perfect search destined to run around in the sands of irrelevance? No—not if we can dress up the content.

## Dressing up the content

The problem of getting consistent, usable search results has already been cracked by your public library. Imagine that you were searching for library books using key-words the way you search URLs on the Internet. There is no telling how long you would need to search before getting the relevant book; often, you may not even know which key-words to use.

But that is not how searching for books in a library has been happening for a long time, nor how you now search for MP3 songs on your personal music player. You search for books in a library using the library catalogue, and for music on your PMP using ID3 tags, which is nothing but metadata (data about data) describing the books and the music respectively.

While search as an activity had been happening in libraries

for a long time, it emerged as a big-ticket application only with the advent of the Internet. With the novelty of this application, there also crept in the implicit assumption that Internet URLs are self-cataloguing. Although that is true to a certain extent when the ‘Title’ and ‘Keyword’ meta tags in HTML are populated correctly, we all know that in the race for search-engine rankings, the most sedate Web designers take extreme liberties with this cataloguing data, making it hard for it to serve any useful purpose. And we have realised that (key-) words that appear directly in the content provide poor and inconsistent self-cataloguing of the content.

Cataloguing or tagging content or documents using standardised schema is the time-tested approach to better, consistent search results in a captive, managed collection. Typical collections can pertain to medical literature, archived news reports, or scientific papers. Certain content just cannot be found without metadata—try searching for MP3 music without ID3 tags.

Content on the Internet is anything but ‘captive’ or managed; besides, it keeps changing in unpredictable ways. Various approaches, such as folksonomy, social indexing, and tagging, are already in vogue to make Internet content search-friendly. If you are active in the blogosphere, you are already using these techniques.

## The needle and the haystack

While searching in uncatalogued content can be a frustrating experience, the search operation becomes more rewarding when we pay as much attention to the content as we do to the search algorithms. In other words, you can find the needle in the haystack provided you thread (tag) the needle and arrange the hay in bales. Once you begin to control and manage your content repository so systematically, you have the beginnings of a Content Management System (CMS).

I hope that this, and the previous article, gave you a relevant perspective on search. 

## References

- [1] [http://en.wikipedia.org/wiki/Vector\\_space\\_model](http://en.wikipedia.org/wiki/Vector_space_model) The vector space model.
- [2] [http://en.wikipedia.org/wiki/Okapi\\_BM25](http://en.wikipedia.org/wiki/Okapi_BM25) The BM25 weighting model.
- [3] [http://xapian.org/docs/intro\\_ir.html](http://xapian.org/docs/intro_ir.html) Technical details of the probabilistic model in a relatively palatable form.
- [4] <http://xapian.org/search> An example of a search engine implementation using the probabilistic model.
- [5] [http://lucene.apache.org/java/2\\_9\\_1/scoring.html](http://lucene.apache.org/java/2_9_1/scoring.html) Explains the Lucene scoring model.

## By: Gurudutt Talgery

The author is a FOSS enthusiast with wide-ranging industry experience. He built and operated an experimental vernacular search engine on the LAMP stack for a couple of years just to appreciate what a difficult job operating an Internet search engine can be. His areas of interest include content management, search engines, and FOSS-in-education.